



University of Asia Pacific

Course title: Software Engineering Lab

Course Code: CSE 314

Sohoj Biniyog

Team Members:

Shahriar Alam Shimul	22201227
Salman Nur Ramim	22201239
H.M. Tahsin Sheikh	22201243

Submitted To :

Noor Mairukh Khan Arnob
Lecturer
Department of Computer Science &
Engineering
University of Asia Pacific

Table of Contents

- Introduction
- Existing Systems and Literature Review
- Proposed System
- Feasibility Study
- Data Collection and Analysis
- Requirement Analysis
- System Design
- UI/UX Design
- Implementation and Development
- Software Quality Assurance (SQA) and Testing
- Project Management
- Results and Discussion
- Conclusion and Future Work
- References
- Appendices

1.1 Background of the Project

Small and Medium Enterprises (SMEs) contribute significantly to Bangladesh's economy but often face difficulties accessing funding from traditional financial institutions. On the other hand, many individuals—especially students, local investors, and diaspora—seek Shariah-compliant investment opportunities. Sohoj Biniyog bridges this gap by providing a halal, transparent, and secure digital platform that connects SME owners with potential investors while ensuring Islamic finance compliance.

1.2 Problem Statement

SMEs struggle to obtain loans due to high interest rates and lack of collateral. Existing crowdfunding or investment platforms in Bangladesh are not Shariah-compliant. Investors lack a trustworthy, transparent system to invest small amounts ethically. There is no digital platform providing real-time updates on SME performance and profit-sharing.

1.3 Objectives of the Project

- Develop a web-based platform for halal investment in SMEs.
- Enable SMEs to upload proposals and investors to fund them securely.
- Ensure Shariah compliance using profit-sharing contracts instead of interest.
- Provide dashboards showing real-time investment updates, profit status, and messaging.
- Build trust using KYC verification, transparency, and secure payment gateways.

1.4 Scope and Limitations

Scope:

- User registration and role selection (Investor / Entrepreneur / Admin)
- SME pitch submission and approval by Admin
- Investment tracking and profit distribution model
- Communication system between entrepreneurs and investors
- Dashboard with visual analytics

Limitations:

- No mobile application in phase 1 (future enhancement)
- Real-time payment gateway simulation only
- Legal approval and certification not fully implemented
- Prototype-level security—full deployment requires advanced encryption

1.5 Target Users / Stakeholders

Stakeholder	Role in System
SME Owners	Submit proposals, request funding
Investors	Invest in halal projects, track profits
Admin	Verify SMEs, approve projects, manage system
Shariah Advisors	Validate Islamic compliance
Government/Regulatory Bodies	Ensure legal and financial compliance

2. Existing Systems and Literature Review

2.1 Overview of Existing Similar Systems

Platform	Description	Limitations
bKash / Nagad / Rocket	Mobile Financial Services used widely for money transfers in Bangladesh.	Not investment platforms; no SME funding or profit-sharing features.
GoFundMe / Kickstarter	Global crowdfunding platforms for creative and startup projects.	Not Shariah-compliant, no profit-sharing models, mainly donation-based.
Islamic Finance Platforms (e.g., Ethis, Yields)	Provide halal investment opportunities following profit-sharing models.	Not available for Bangladeshi SMEs; international investment barriers.
Banks and Microfinance Institutions	Provide business loans to SMEs with interest.	Interest-based (riba), strict documentation, collateral required.

2.2 Comparative Analysis

Features	Traditional Banks	Crowdfunding (Kickstarter)	Islamic Investment Platforms (Ethis)	Sohoj Biniyog (Proposed)
Shariah Compliant	✗	✗	✓	✓
Investment Type	Loan (Interest)	Donation/Reward	Profit Sharing	Profit Sharing

Features	Traditional Banks	Crowdfunding (Kickstarter)	Islamic Investment Platforms (Ethis)	Sohoj Biniyog (Proposed)
Minimum Investment	High	Low	Medium	Very Low (₳500-1000)
SME Focus	Limited	Creative Projects	Real Estate/Business	Local SMEs in BD
Transparency & Dashboard	Limited	Medium	High	High
Real-Time Updates	✗	✗	✓	✓
Investor–Entrepreneur Direct Contact	✗	✗	✗	✓

2.3 Literature Review and Related Research

Previous studies show:

- Islamic finance models such as Mudarabah (profit-sharing) and Musharakah (partnership) are effective alternatives to interest-based loans.
- Research papers highlight that 67% of Bangladeshi SMEs struggle with financing due to high-interest rates and lack of collateral.
- Studies on fintech adoption in Bangladesh indicate a rise in digital transactions due to mobile banking, suggesting high potential for an online halal investment platform.
- Literature also emphasizes trust, transparency, and Shariah compliance as the core factors influencing user acceptance of Islamic investment platforms.

2.4 Key Gaps Identified

- **No existing Bangladeshi platform combines** SME funding + halal finance + digital transparency.
- **Crowdfunding platforms do not provide** profit-sharing or contract-based investment models.
- **Lack of** real-time dashboards and investor–SME communication systems.
- **Investors lack** trust due to fraud, misinformation, and absence of verification systems.

3. Proposed System

3.1 Overview of the Proposed Solution

Sohoj Biniyog is a web-based halal investment platform that connects investors with SME entrepreneurs in Bangladesh. The system ensures Shariah compliance by using *profit-sharing agreements* instead of interest-based loans. It provides a secure and transparent environment with real-time dashboards, project tracking, and investor–entrepreneur communication features.

3.2 Key Features and Functionalities

For Investors:

- Registration and role selection as an investor
- Browse and filter SME project proposals
- View investment amount, expected ROI, and risk level
- Invest digitally using mobile financial services (bKash, Nagad)
- Real-time dashboard: investment status, returns, profit history
- Receive notifications about profit cycles, project updates

For SME Entrepreneurs:

- Sign up and submit business proposals
- Add investment goals, business plans, required capital
- Upload relevant documents for verification
- Communicate with potential investors
- Receive funds only after admin approval
- Track investor interest and funding progress

For Admin:

- Approve or reject business proposals
- Verify identities using KYC and documentation
- Manage investment agreements and disputes
- Monitor transactions and system security

Additional Features (Nice-to-Have):

- User badges (Verified / Trusted)
- Multilingual interface (English & Bangla)
- Automated PDF agreement generation
- AI-based SME recommendations for investors
- Community forum for investor discussions

3.3 Innovation and Technology Stack

Layer	Technologies	Purpose
Frontend	HTML, CSS, JavaScript	User Interface
Backend Framework	Python (Django)	Business Logic & API
Database	MySQL	User, SME & Investment Data
Hosting	Railway / AWS	Cloud Deployment
Payment Integration	bKash, Nagad, Rocket APIs	Digital Transactions
Authentication	Django Auth + 2FA	Secure Login
Security Layer	HTTPS, KYC Verification	Prevent Fraud & Data Breach

3.4 System Architecture Overview (Text-Based Description)

User Layer (Investor/Entrepreneur/Admin) →
Frontend Interface (Browser) →
Backend Server (Django Framework) →
Database (MySQL) →
Payment Gateways & External APIs (bKash/Nagad)

Data Flow:

1. User submits login/request →
2. Django validates and processes →
3. Data stored/retrieved from MySQL →
4. Backend responds with results →
5. Real-time updates rendered on Dashboard.

3.5 Expected Outcomes

- ✓ A functional, user-friendly halal investment portal.
- ✓ Increased accessibility for SMEs to get funding.
- ✓ Transparent, safe investment process for individuals.
- ✓ Establishment of trust using Islamic finance principles.
- ✓ Foundation for a scalable fintech startup.

4. Feasibility Analysis

4.1 Technical Feasibility:

- **Frontend Technologies:** HTML, CSS and Java Script.
- **Backend Framework:** Python, Django.
- **Database System:** MySQL.
- **Hosting:** Railway (free/low-cost hosting).
- **API Integrations:** Mobile financial service, APIs like **bKash, Nagad, Rocket, Upay** will be integrated for payment functionality.
- **Security & Compliance:** Two-factor authentication.

4.2 Economic Feasibility:

Initial Development Cost: Minimal due to open-source tools and student-led development.

Revenue Model:

- Service fee from successful investments (e.g., 2–3%)
- Subscription or tiered services for SMEs (e.g., verified listing, promotion)
- Affiliate or partnership programs with halal finance institutions

Break-even Potential: With only 500 monthly active users investing ₳1000/month, the platform can generate self-sustaining income.

4.3 Operational Feasibility:

- **User Adoption:** Increasing smartphone and mobile internet usage in Bangladesh make it easy to reach the target audience.
- **Ease of Use:** The UI/UX will focus on simplicity, multilingual support (Bangla + English), and clear investment terms.
- **Human Resources:** Development and maintenance can initially be handled by a small team (3–5 people), scaling later as needed.
- **Training & Support:** In-app educational modules and support chatbots will reduce the need for constant manual support.

4.4 Legal Feasibility:

- Profit-sharing contracts will be used instead of interest-based returns.
- A Shariah advisory board or validator group will oversee SME listings and investment models.
- Transactions must comply with Bangladesh Bank guidelines for mobile financial services (MFS).
- Proper documentation, identity verification (NID/passport), and AML/KYC protocols will be followed.

4.5 SWOT Analysis:

Strengths	Weaknesses
➤ Real problem in Bangladesh	➤ Trust issues between strangers
➤ Two-sided win-win model	➤ Initial low user base
➤ Simple tech stack (easy to build)	➤ No full legal framework for equity funding

Opportunities	Threats
❖ Can grow into a real startup	❖ Fraud risk from fake pitches
❖ Potential to partner with NGO	❖ Regulatory or legal changes
❖ Can add mobile app for wider reach	❖ Larger competitors may enter later

5.Data Collection and Analysis

Survey Questionnaires

Survey link:

<https://docs.google.com/forms/d/e/1FAIpQLSfAiYievoA-oTadmZZVBV0ZFKNVW4ld2Pzv2f0UPqyqnkfaA/viewform?usp=header>

Demographic Section:

- Age: [Under 18], [18–25], [26–35], [36–45], [46+]
- Occupation: [Student], [Business Owner], [Service Holder], [Other]

Multiple Choice:

1. Have you ever invested in any platform?
 - Yes / No
2. Which investment range do you feel comfortable with?
 - Less than ₳500, ₳500 – ₳1,000, ₳1,000 – ₳5,000, More than ₳5,000
3. How often would you prefer to invest?
4. What motivates you to consider Halal investments?
 - Ethics, Profit, Simplicity, Religious Value

Likert Scale (1=Strongly Disagree, 5=Strongly Agree):

How much do you agree with the following statements?

- I am comfortable using mobile/web apps for financial transactions.

- I am interested in investing in ethical (halal) businesses.
- I trust platforms that show transparency and real-time updates.
- I would prefer investing in small, verified local businesses (SMEs).
- I am more likely to invest if the platform provides proper Islamic guidance.

Open-ended:

- What features or services would you like to see in a small investment platform like Sohoj Biniyog?
- What concerns would stop you from using such a platform?

Interview Questionnaires

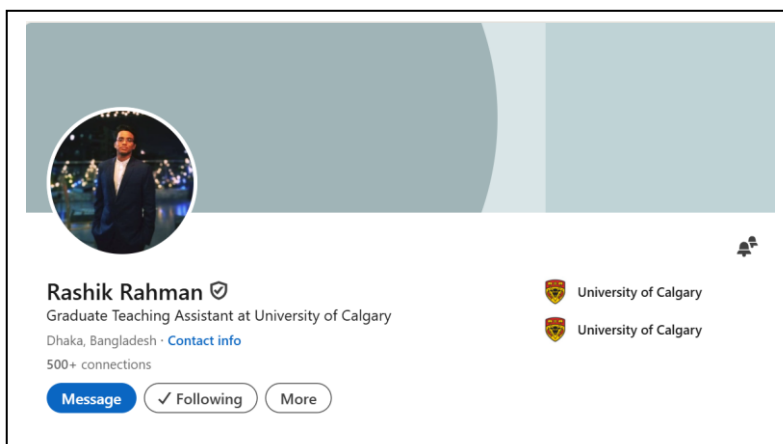
Semi-Structured Questions:

1. What is your first impression of a platform like “Sohoj Biniyog”.
2. What would motivate you to trust a platform like "Sohoj Biniyog"?
3. Which tech stack (language/frameworks/tools) would you personally use to build a scalable MVP for this platform, and why?
4. What would be your biggest concern if you were investing in a project like this — from both a tech and business angle?
5. What advice would you give to a student team trying to turn this project into a real product or startup?

5.2 Interview Process

Interviewee:

- **Name:** Rashik Rahman
- **Role:** University Lecturer



Thematic Analysis:

1. What would motivate you to trust a platform like "Sohoj Biniyog"?
It's a great idea. It helps students and small businesses connect in a halal way.
Many people need this.

2. What would motivate you to trust this platform?
I need to see verified SMEs, secure contracts and real-time updates.

3. Which tech stack (language/frameworks/tools) would you personally use to build a scalable MVP for this platform, and why?

I'd use Django for backend, React or Flutter for frontend, PostgreSQL for database, and Railway or AWS for hosting.

4. What would be your biggest concern if you were investing in a project like this — from both a tech and business angle?

Tech concern: data security. Business concern: user trust and fake SMEs.

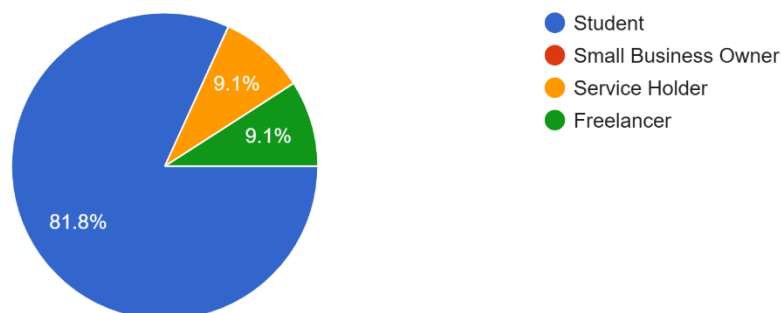
5. What advice would you give to a student team trying to turn this project into a real product or startup?

Start small, talk to users early, build trust, and keep it simple. Get a mentor or advisor if possible.

5.3 Survey Results & Summary

We surveyed 11 participants to evaluate challenges in our project. Key findings are visualized below:

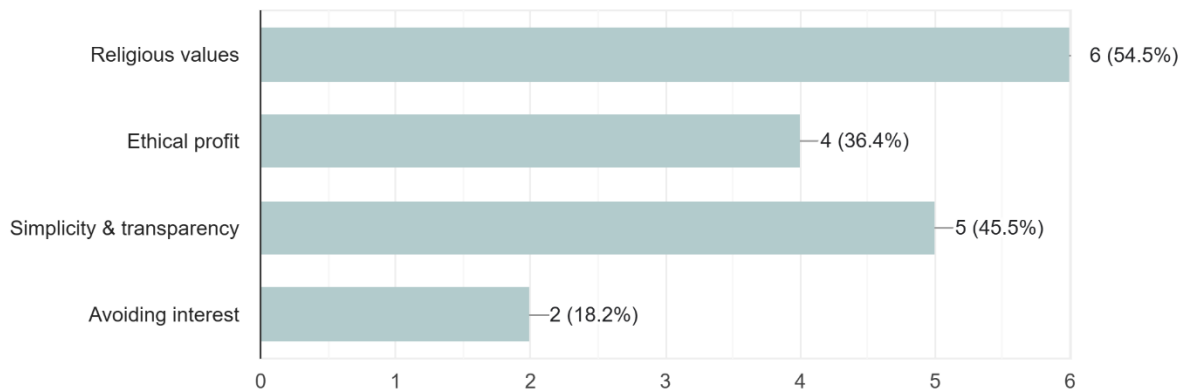
Occupation
11 responses



The majority (81%) of respondents were students, with the remaining being service holders (9%) and freelancers (9%). This shows that students are highly engaged and interested in investment opportunities.

What motivates you to consider Halal investments? (Select all that apply)

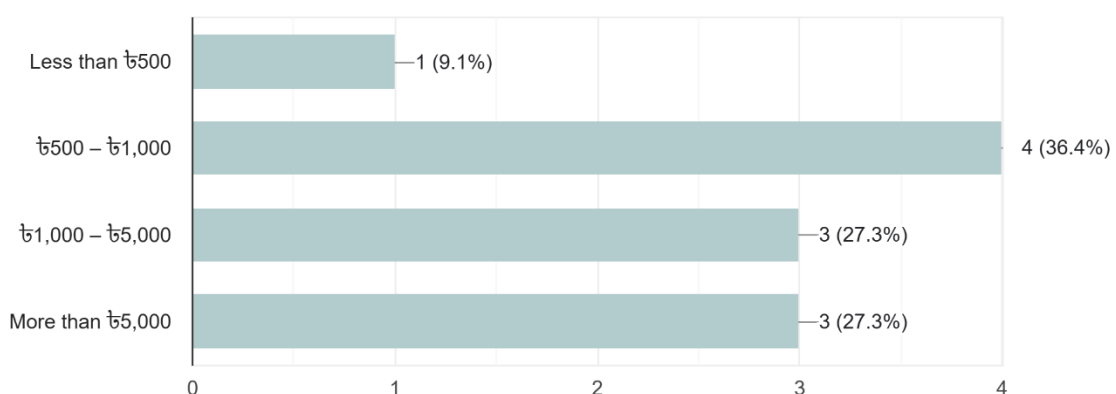
11 responses



Respondents cited religious values and ethical practices as the top motivations for choosing halal investments. This highlights the importance of maintaining visible Shariah compliance, transparency, and ethical models in the platform to meet user expectations and build trust.

Which investment range do you feel comfortable with? (Select all that apply)

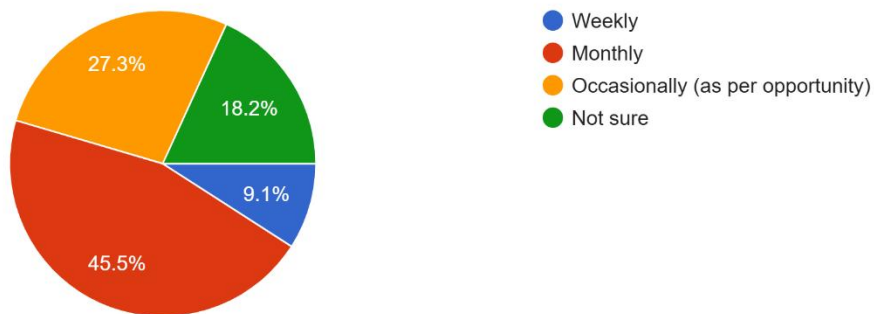
11 responses



Most participants preferred to invest in the range of **₹500–₹1,000**, with some willing to go up to **₹5,000**. This confirms the need for a **low-barrier entry system**, making investing accessible to students and the middle-income group.

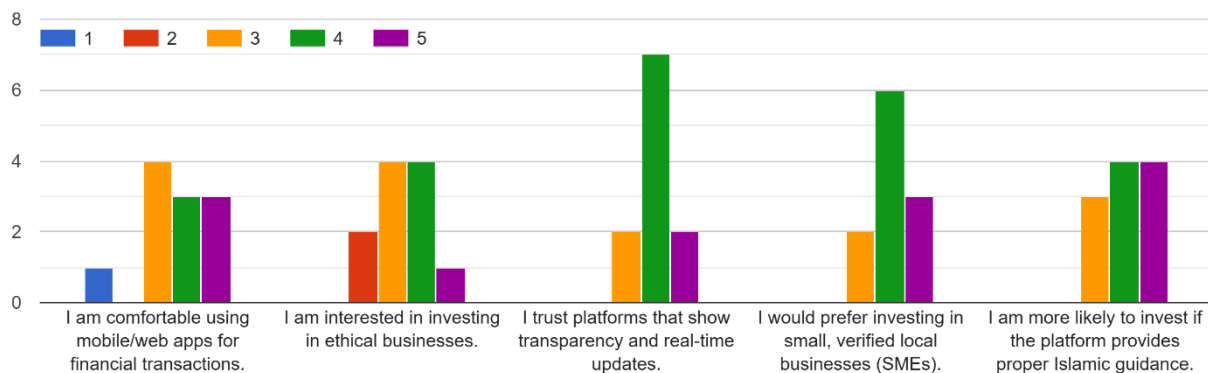
How often would you prefer to invest?

11 responses



A significant portion of users preferred **monthly or occasional** investments rather than frequent or high-volume contributions.

How much do you agree with the following statements? Please click that if your answer is: Rate from, Strongly Disagree= 1, Disagree = 2, Neutral = 3, Agree = 4, Strongly Agree = 5



Participants largely agreed (scoring 4 or 5) on comfort with mobile/web apps, trust in transparent platforms, and willingness to invest in small, local SMEs.

What features or services would you like to see in a small investment platform like Sohoj Biniyog?

11 responses

Easy money withdrawal
Halal investment,good security.
Give KYC verification for user & verify their info by direct call
Give proper guidelines about policy
Weekly updates
Ability to know where my money is being spent or used to earn the interest
No
More profit
Customer support

What concerns might stop you from using such a platform?

11 responses

I will need strong verification and transparency before investing.
Trust
Transaction problems
Trust issues
Money londering or lost of trust
Lack of investment
Transaction failure
Whether it is halal or haram
No

Key Trends and Findings

Observation	Result
Preferred investment amount	₹500 – ₹1000 by majority
Motivation for halal investment	Religious values and ethics were top reasons
Investment frequency	Most preferred monthly or occasional investment
Trust in transparency	Majority scored 4 or 5 on need for real-time updates
Comfort using apps	Most were comfortable with digital platforms for transactions

User Concerns (From Open-ended Questions)

- Fear of fraud or misuse of money
- Trust issues with SMEs
- Need for verified business documentation
- Desire for profit calculation transparency

5.4 Interpretation of Results

- The majority of respondents are ready to invest small amounts if the platform is transparent and Shariah-compliant.
- Students and young investors are the primary target audience.
- Religious trust, transparency, and security are the most important factors for platform adoption.
- Real-time updates, low barriers to entry, and verified SMEs are essential to gain investor confidence.

6. Requirement Analysis

6.1 Functional Requirements

Requirement Type	Description
User Registration & Login	Users must be able to create accounts and log in securely.
Role Selection	Users can select roles: Investor, Entrepreneur, or Admin.

Requirement Type	Description
SME Proposal Submission	Entrepreneurs can submit business pitches with funding goals and descriptions.
Admin Approval System	Admin reviews and approves/rejects proposals before publishing.
Browse & Search Projects	Investors can browse, filter, and search SME projects.
Investment Feature	Investors can select a project, invest a chosen amount, and receive confirmation.
Profit-Sharing Contract Generation	System auto-generates contracts based on investment ratio and profit rules .
Dashboard for Investors	Shows total investment, profit earned, pending payments, project updates.
Dashboard for Entrepreneurs	Shows funding progress, investors list, project status.
Messaging System	Secure chat between investors and entrepreneurs.
Notifications & Alerts	Email/SMS or in-app notifications for new investments, project approvals, and profits.
Admin Panel	Admin can manage users, projects, investments, complaints, and system settings.

6.2 Non-Functional Requirements

Category	Requirement
Performance	System should support 1000+ concurrent users; response time < 3 seconds.
Security	SSL encryption, HTTPS, two-factor authentication, secure login.
Reliability	System uptime should be at least 99.9%.
Scalability	Should support future growth (mobile app, more users, more transactions).
Usability	User-friendly UI/UX; multilingual support (English + Bangla).

Category	Requirement
Backup & Recovery	Database backup should occur automatically every 24 hours.
Compliance	Follows Shariah finance principles and KYC/AML regulations.
Accessibility	Platform should be usable by visually impaired users (screen reader support).

6.3 Prioritization of Requirements

Priority Level	Features
Must-Have	Registration & Login, Role Selection, SME Proposal Submission, Investment Functionality, Admin Approval, Messaging System, Dashboard (Investor & Entrepreneur), Secure Payment Integration.
Nice-to-Have	AI-based SME recommendations, PDF export of investment contracts, User verification badges, Notifications via SMS/Email, Chatbot support, Mobile application.

6.4 Use-Case Scenarios (Text Description)

Use Case 1: Investor Invests in a Project

- **Actor:** Investor
- **Precondition:** Investor is logged in
- **Flow:**
 1. Investor browses available projects
 2. Selects a project and clicks “Invest”
 3. Enters investment amount
 4. System processes payment
 5. Contract is generated and stored
 6. Investor sees investment on dashboard

Use Case 2: SME Submits Business Proposal

- **Actor:** Entrepreneur
- **Precondition:** Must be logged in and profile completed
- **Flow:**
 1. Visit “Submit Proposal” page

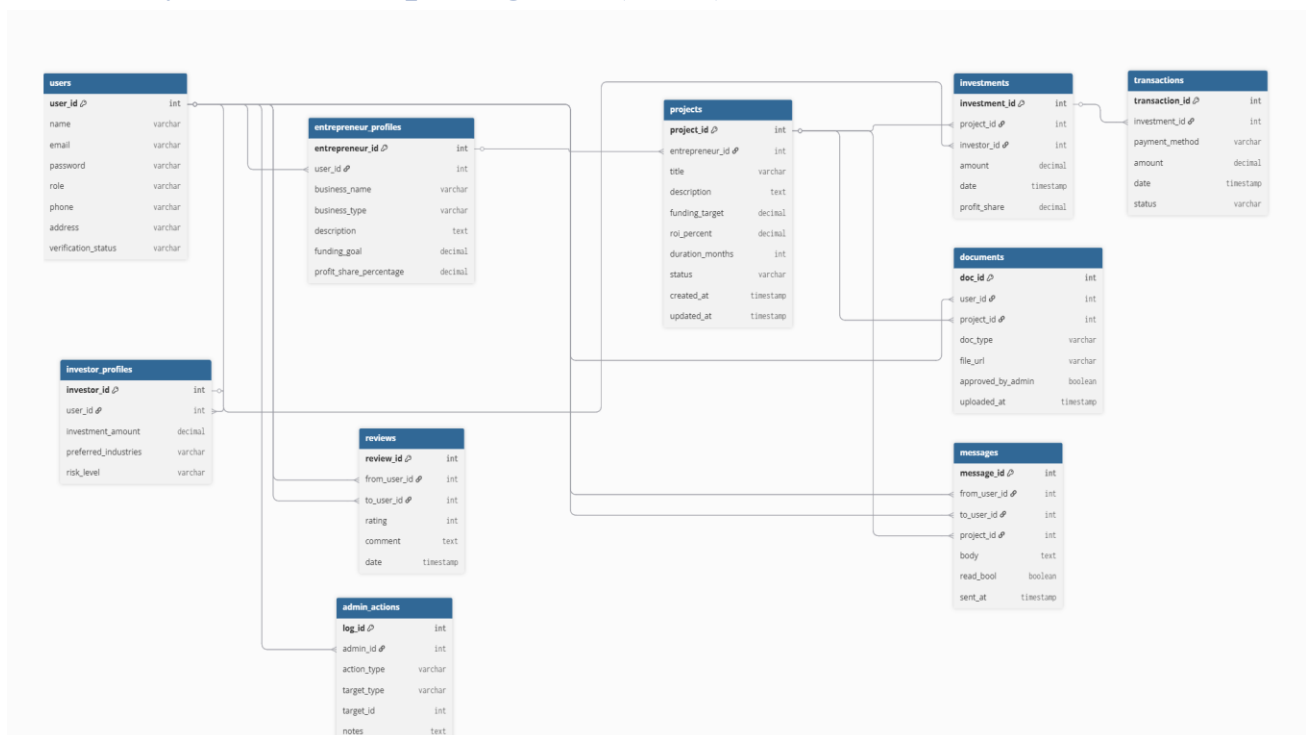
2. Enter business details, funding goal, profit-sharing percentage
3. Upload documents (trade license, NID)
4. Admin reviews and approves/rejects
5. If approved, project becomes visible to investors

Use Case 3: Admin Verifies and Approves Projects

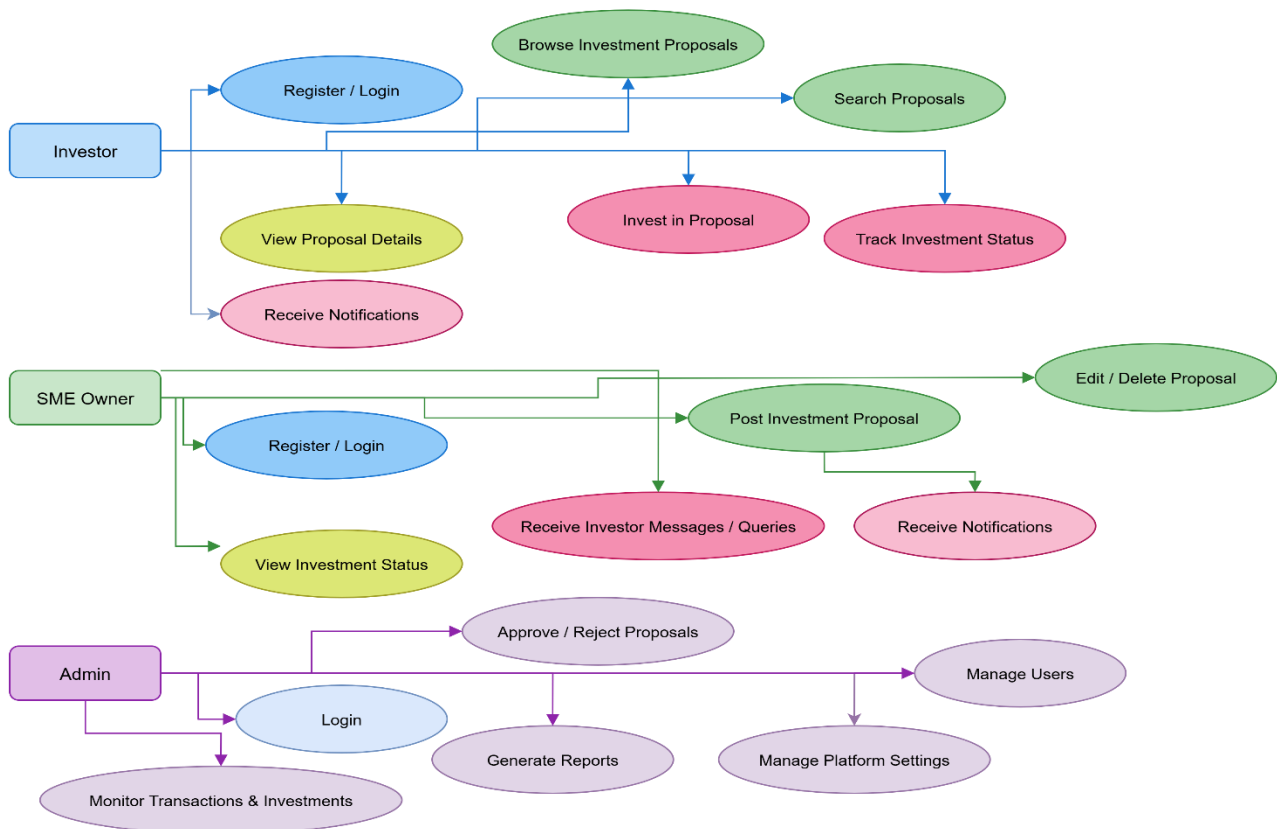
- **Actor:** Admin
- **Flow:**
 1. Log into Admin Panel
 2. View list of pending SME proposals
 3. Verify documents and information
 4. Approve or reject the proposal
 5. Approved projects go live on the platform

7. System Design

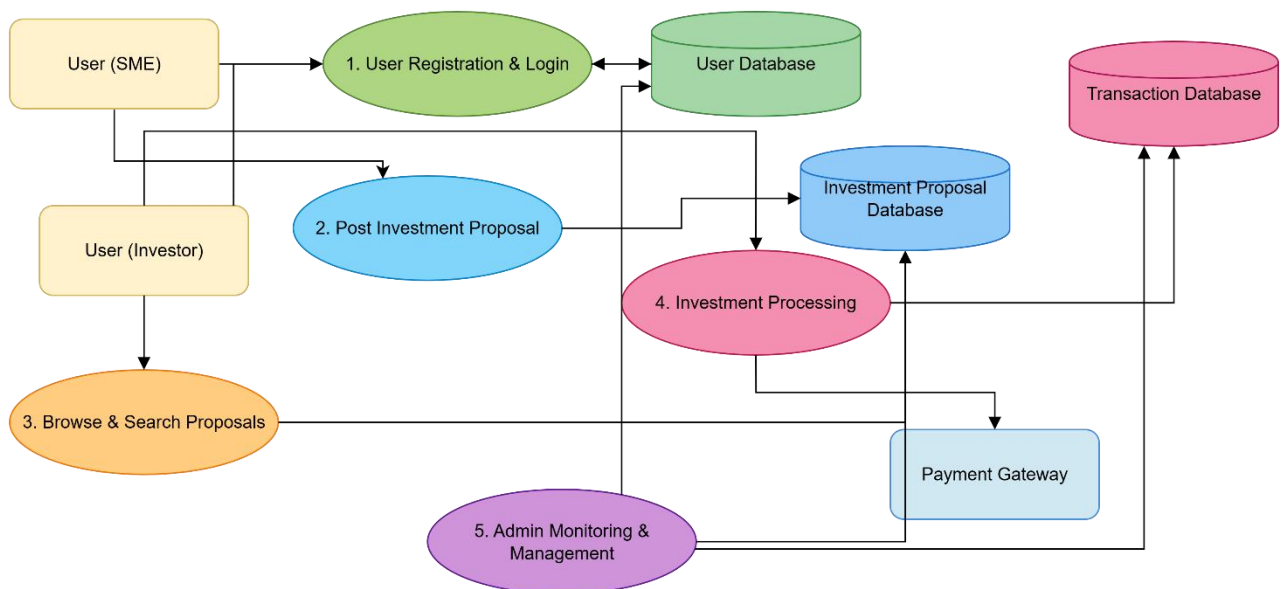
7.1 Entity-Relationship Diagram (ERD)



7.2 Use-Case Diagram



7.3 Data Flow Diagram (Level 1 DFD)



8. UI/UX Design (Figma Prototype)

8.1 Overview of the Design Objective

The goal of the UI/UX design for *Sohoj Biniyog* is to build a simple, clean, and user-friendly interface that allows investors and SME owners to interact smoothly. The design prioritizes:

- Ease of navigation
- Trust and transparency
- Minimalistic and modern layout
- Alignment with Islamic finance values

8.2 Design Process and Tools Used

Step	Description
Research & Planning	Studied similar platforms like crowdfunding websites and Islamic finance portals.
Wireframing	Created low-fidelity wireframes using Figma to define layout and structure.
Prototype Development	Built high-fidelity desktop views using a 12-column grid for alignment.
Component Reusability	Reused card designs, buttons, navbar, and dashboards across multiple pages.
Testing & Feedback	Improved design after team review and minor user testing.

Tools Used:

- Figma (UI Design & Prototyping)
- Iconify and Boxicons (Icons)
- Pinterest (UI Inspiration & Visual References)

8.3 Layout Structure (Desktop Version)

- **Navbar:** Logo, Home, Funded Campaign, About Us, Contact, Login/Register buttons.
- **Hero Section:** Headline, call-to-action button (“Invest Now”, “Submit a Proposal”).

- **Project Cards:** Business title, funding required, minimum investment, status bar.
- **Dashboard (Investor):** Total investments, active projects, expected profit, messages.
- **Dashboard (Entrepreneur):** Funding progress, number of investors, proposal status.
- **Footer:** Social links, privacy policy, contact details.

8.4 Key Components Designed

Component	Purpose
Navigation Bar	Quick access to major website sections
Project Card	Displays business details and investment status
Funding Progress Bar	Shows percentage of money raised vs. goal
Buttons (Primary/Secondary)	For actions like Invest, Submit, Login
Chat Interface	Messaging between users
Sidebar (Dashboard)	Displays navigation items for logged-in users

8.5 Responsiveness and Auto Layout

- All UI elements were designed using auto-layout in Figma.
- Supports scaling for larger/lower resolution screens.
- Future plan includes designs for tablet and mobile versions.

8.6 Prototyping and Interactive Navigation

- Button interactions and transitions were added in Figma.
- Clicking "Invest Now" opens project detail page.
- SME submission form is fully navigable.
- Dashboard screens switch between tabs using clickable sidebars.

8.7 Challenges Faced and Solutions

Challenge	Solution
Maintaining design consistency	Created reusable components (buttons, cards, icons)

Challenge	Solution
Balancing professionalism with simplicity	Used minimal colors, soft shades, and readable fonts
Ensuring Islamic compliance visually	Avoided interest-related symbols; used Islamic-friendly fonts and icons

8.8 Future UI/UX Improvements

- Mobile App UI for Android & iOS
- Dark mode support
- Multi-language toggle (Bangla/English)
- Improved accessibility (screen reader support, high contrast mode)
- Animated graphs for investment statistics

9. Implementation and Development

9.1 Front-End Development

Technologies Used:

- HTML – For structure
- CSS – For styling & layout
- JavaScript – For interactivity and dynamic content
- Responsive Design – Ensured adaptability across different screen sizes
- Reusable Components: Navigation bar, buttons, cards, input forms

Modules Developed:

Module	Description
Home Page	Introductory section with CTA buttons like <i>Invest Now</i> , <i>Submit Proposal</i>
Project Listing Page	Displays available SME projects with filters (category, budget, profitability)
Login/Registration Pages	Separate forms for investors, entrepreneurs, and admin

Module	Description
SME Proposal Form	Entrepreneurs submit business plan, funding goal, profit-sharing percentage
Dashboard (Investor)	Displays total investments, pending profits, ongoing projects
Dashboard (Entrepreneur)	Shows funding progress, investors list, approval status
Messaging Interface	Chat box between investors and entrepreneurs

9.2 Back-End Development

Framework Used: Django (Python)

Key Features Implemented:

- User authentication system with role-based access
- CRUD operations for projects and investments
- MySQL database integration using Django ORM
- Session management and password hashing
- Admin panel for verifying pitches, users, and transactions

9.2.1 Basic Page Implementation

- Home, About Us, Contact, Projects, Dashboard pages built using Django templates
- URL mapping configured for each view in urls.py
- Views created in views.py to render templates and handle logic

9.2.2 Database Integration

- models.py used to define tables such as User, Project, Investment, Contract
- Django ORM handles database migrations and queries automatically

9.2.3 API Endpoints and Logic

Endpoint	Method	Description
/register/	POST	Creates a new user
/login/	POST	Authenticates user and sets session

Endpoint	Method	Description
/projects/	GET	View all approved projects
/submit_project/	POST	SME submits a project proposal
/invest/<project_id>/	POST	Investor makes an investment
/dashboard/	GET	Shows personalized dashboard based on role

9.3 Front-End and Back-End Integration

- Data passed through Django context and templates
- AJAX used in some parts for live updates without page reload
- CSRF tokens implemented to secure form submissions
- Dynamic project listings fetched from the database and displayed in real-time

9.4 Tools, Frameworks, and Libraries Used

Domain	Tools/Frameworks
Frontend	HTML, CSS, JavaScript
Backend	Python, Django
Database	MySQL, db.sqlite3
Design	Figma, Iconify, Boxicons
Version Control	Git & GitHub
Others	Django Authentication, Django ORM

9.5 GitHub Commits and Team Contributions

Commits		
main	All users	All time
Commits on Oct 28, 2025		
Merge branch 'main' of https://github.com/Tahsin2201243/SohojWeb	5c18b54	<>
Initial commit	1983bad7	<>
Commits on Oct 24, 2025		
Update README.md	ec5a155a	<>
Update README.md	8981464	<>
Update README.md	b7409ba2	<>
Create README.md	857d54b	<>
Commits on Oct 20, 2025		
Selenium testing done for all the pages of the project	915d8952	<>
ui upgrade, backend modified, updated views logic	e140a2b	<>
Commits on Sep 29, 2025		
Login Registration added	2c22785	<>
Commits on Sep 28, 2025		
Initial project commit	ead7907c	<>
Commits on Sep 25, 2025		
Modified CSS	4027f65	<>
Commits on Sep 22, 2025		
Navbar Updated	ca5e78d2	<>
Frontend Modified	47ab299	<>
Commits on Sep 21, 2025		
html files created	3d7553c7	<>
Commits on Sep 20, 2025		
Backend for some pages	2553ac9f	<>
requirement.txt file created	7596da2	<>
Commits on Sep 16, 2025		
Initial commit		

Contributors

Period: All

Contributions: Commits

Contributions per week to main, excluding merge commits

Commits over time

Weekly from 14 Sept 2025 to 26 Oct 2025





10. Software Quality Assurance (SQA) and Testing

10.1 Introduction to Testing Strategy

To ensure the reliability, security, and performance of *Sohoj Biniyog*, both **manual and automated testing approaches** were used.

The primary objective of testing was to ensure that:

- All functional requirements work properly
- No critical bugs exist in user registration, login, investment, and payment workflows
- The system is secure and user-friendly

10.2 Selenium Setup (Automation Testing)

Selenium WebDriver with Chrome was used for basic automation testing of UI components.

Steps:

1. Installed Selenium and Chrome WebDriver
2. Wrote Python scripts to automate login, navigation, and form submissions
3. Executed scripts to test real-time user actions

10.3 Test Cases and Scenarios

Test Case ID	Test Scenario	Expected Output	Result
TC01	User Registration	New user registered	Passed
TC02	Invalid Login Attempt	Error message displayed	Passed
TC03	SME submits project proposal	Proposal saved & sent for admin approval	Passed
TC04	Admin approves project	Status updated to “Approved”	Passed
TC05	Investor invests in a project	Transaction completed & dashboard updated	Passed
TC06	Empty fields in form submission	Validation error shown	Passed
TC07	Unauthorized access attempt	Redirect to login page	Passed
TC08	Messaging feature	Message sent and received successfully	Passed

10.4 Results of Test Execution

- All major functional requirements worked as expected
- No high-priority bugs were found
- Minor UI alignment issues were resolved
- Application handled incorrect inputs and empty fields properly

Register_sme

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

# Example:
driver = webdriver.Chrome()
driver.maximize_window()
driver.get("http://127.0.0.1:8000/register/") # change if needed

wait = WebDriverWait(driver, 15)

# Scroll down slowly
driver.execute_script("window.scrollTo(0, document.body.scrollHeight / 2);")
time.sleep(1)
driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
print("Scrolled through the page")

# Select Business Owner role
role_business = wait.until(EC.element_to_be_clickable((By.ID, "roleBusiness")))
driver.execute_script("arguments[0].scrollIntoView(true);", role_business)
time.sleep(0.5)
driver.execute_script("arguments[0].click();", role_business)
print("Selected role: Business Owner")

# Fill in registration form
username = wait.until(EC.visibility_of_element_located((By.NAME, "username")))
email = driver.find_element(By.NAME, "email")
password = driver.find_element(By.NAME, "password")
confirm = driver.find_element(By.NAME, "confirm")

username.send_keys("business_user1234")
email.send_keys("business_user1234@example.com")
password.send_keys("TestPassword123")
confirm.send_keys("TestPassword123")
print("Filled in registration details")

# Click Register
submit_btn = driver.find_element(By.ID, "submitBtn")
driver.execute_script("arguments[0].scrollIntoView(true);", submit_btn)
```

```
✓ Scrolled through the page
✓ Selected role: Business Owner
✓ Filled in registration details
✓ Clicked Register button
```

Apply for investment:

```
Click to add a breakpoint to webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
import os

# --- Setup ---
driver = webdriver.Chrome()
driver.maximize_window()
wait = WebDriverWait(driver, 15)

# --- Step 1: Login as Business Owner ---
driver.get("http://127.0.0.1:8000/login/")

username = wait.until(EC.visibility_of_element_located((By.NAME, "username")))
password = driver.find_element(By.NAME, "password")

username.send_keys("business_user1234") # Use registered business owner credentials
password.send_keys("TestPassword123")

login_button = driver.find_element(By.CSS_SELECTOR, "button[type='submit']")
login_button.click()

# --- Step 2: Go to Apply for Investment page ---
driver.get("http://127.0.0.1:8000/apply/") # Adjust URL as needed
wait.until(EC.visibility_of_element_located((By.CSS_SELECTOR, "h2.text-center")))
print("Apply for Investment page loaded")
time.sleep(1)

# --- Step 3: Fill in the form ---
business_name = driver.find_element(By.NAME, "name")
description = driver.find_element(By.NAME, "description")
location = driver.find_element(By.NAME, "location")
website = driver.find_element(By.NAME, "website")
documents = driver.find_element(By.ID, "id_documents")
terms_checkbox = driver.find_element(By.ID, "id_terms")

business_name.clear()
business_name.send_keys("My Business Test")
description.clear()
```

Post for investment:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
import os

# --- Setup ---
driver = webdriver.Chrome()
driver.maximize_window()
wait = WebDriverWait(driver, 15)

# --- Step 1: Login as Business Owner ---
driver.get("http://127.0.0.1:8080/login/") # Adjust URL

username = wait.until(EC.visibility_of_element_located((By.NAME, "username")))
password = driver.find_element(By.NAME, "password")

username.send_keys("business_user1234") # Business owner credentials
password.send_keys("TestPassword123")

login_btn = driver.find_element(By.CSS_SELECTOR, "button[type='submit']")
login_btn.click()

# --- Step 2: Navigate to Post Campaign page ---
driver.get("http://127.0.0.1:8080/post/") # Adjust URL if needed
wait.until(EC.visibility_of_element_located((By.CSS_SELECTOR, "h2.text-center")))
print("✅ Post Campaign page loaded")
time.sleep(1)

# --- Step 3: Fill the form ---
# Select approved business
business_select = wait.until(EC.element_to_be_clickable((By.ID, "business")))
business_select.click()
business_option = business_select.find_element(By.CSS_SELECTOR, "option") # selects first approved business
business_option.click()

# Fill text fields
driver.find_element(By.NAME, "title").send_keys("My Test Campaign")
driver.find_element(By.NAME, "description").send_keys("This is a test campaign for Selenium automation.")
driver.find_element(By.NAME, "target_amount").send_keys("100000")
driver.find_element(By.NAME, "min_investment").send_keys("1")
driver.find_element(By.NAME, "start_date").send_keys("2-10-2025")
driver.find_element(By.NAME, "end_date").send_keys("2-12-2029")

# Upload cover image (optional)
cover_input = driver.find_element(By.NAME, "cover_image")
file_path = os.path.abspath("image.jpg") # Replace with an actual test image path
cover_input.send_keys(file_path)
time.sleep(1) # wait for preview to load
```

Register_investor:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

# ✅ Setup
driver = webdriver.Chrome()
driver.maximize_window()
driver.get("http://127.0.0.1:8080/register/") # change to your local URL

wait = WebDriverWait(driver, 15)

# ✅ Scroll down slowly
driver.execute_script("window.scrollTo(0, document.body.scrollHeight / 2);")
time.sleep(1)
driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
print("✅ Scrolled through the page")

# ✅ Select Investor role
role_investor = wait.until(EC.element_to_be_clickable((By.ID, "roleInvestor")))
driver.execute_script("arguments[0].scrollIntoView(true);", role_investor)
time.sleep(0.5)
driver.execute_script("arguments[0].click();", role_investor)
print("✅ Selected role: Investor")

# ✅ Fill in registration form
username = wait.until(EC.visibility_of_element_located((By.NAME, "username")))
email = driver.find_element(By.NAME, "email")
password = driver.find_element(By.NAME, "password")
confirm = driver.find_element(By.NAME, "confirm")

username.send_keys("investor_user1234")
email.send_keys("investor_user1234@example.com")
password.send_keys("TestPassword123")
confirm.send_keys("TestPassword123")
print("✅ Filled in registration details")

# ✅ Click Register button
submit_btn = driver.find_element(By.ID, "submitBtn")
driver.execute_script("arguments[0].scrollIntoView(true);", submit_btn)
time.sleep(0.5)
driver.execute_script("arguments[0].click();", submit_btn)
print("✅ Clicked Register button")

# ✅ Optional pause to visually confirm
time.sleep(3)
driver.quit()
```

```
✅ Scrolled through the page
✅ Selected role: Investor
✅ Filled in registration details
✅ Clicked Register button
```

Investment_campaign:

```
from selenium import webdriver
# Click to add a breakpoint liver-common by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

# --- Step 1: Setup ---
driver = webdriver.Chrome()
driver.maximize_window()
wait = WebDriverWait(driver, 15)

# --- Step 2: Log in as investor ---
driver.get("http://127.0.0.1:8000/login/") # adjust if your URL differs

# Fill login form
wait.until(EC.visibility_of_element_located((By.ID, "username"))).send_keys("investor_user1234")
driver.find_element(By.ID, "password").send_keys("TestPassword123")
driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
print("✅ Logged in as Investor")

# --- Step 3: Go to campaign invest page ---
# Example: http://127.0.0.1:8000/campaign/1/invest/
driver.get("http://127.0.0.1:8000/campaign/10/invest/")
print("✅ Opened campaign investment page")

# --- Step 4: Wait for form to load ---
amount_input = wait.until(EC.visibility_of_element_located((By.ID, "amount")))

# --- Step 5: Fill investment amount ---
amount_value = "5000" # you can adjust this as needed
amount_input.clear()
amount_input.send_keys(amount_value)
print(f"✅ Entered investment amount: {amount_value}")

# --- Step 6: Submit the form ---
invest_button = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Invest Now')]")))
invest_button.click()
print("✅ Clicked 'Invest Now' button")

# --- Step 7: Wait for confirmation ---
# Wait for success alert or page change
try:
    success_message = wait.until(EC.visibility_of_element_located(
        (By.XPATH, "//div[contains(@class, 'alert-success')]")
    ))
    print("✅ Investment submitted successfully!")
except:
    print("⚠️ No success message found. Check if investment was submitted or campaign is closed.")

time.sleep(2)
driver.quit()
```

- ✅ Logged in as Investor
- ✅ Opened campaign investment page
- ✅ Entered investment amount: 5000
- ✅ Clicked 'Invest Now' button

Campaign_details:

```
driver.maximize_window()
wait = WebDriverWait(driver, 10)

try:
    # --- Step 1: Login ---
    print("🔑 Navigating to login page...")
    driver.get(LOGIN_URL)

    # Wait for username and password fields
    username_input = wait.until(EC.visibility_of_element_located((By.NAME, "username")))
    password_input = wait.until(EC.visibility_of_element_located((By.NAME, "password")))

    # Fill in credentials
    username_input.send_keys(USERNAME)
    password_input.send_keys(PASSWORD)
    password_input.submit()
    print(f"✅ Logged in as {USERNAME}")
    time.sleep(2)

    # --- Step 2: Go to Campaign Details page ---
    driver.get("http://127.0.0.1:8000/campaign/11/")
    print("✅ Opened campaign details page")
    time.sleep(1)

    # --- Step 3: Wait for campaign info to load ---
    campaign_card = wait.until(EC.visibility_of_element_located((By.CSS_SELECTOR, ".campaign-card")))
    print("✅ Campaign card loaded")
    time.sleep(1)

    # --- Step 4: Verify campaign title & business name ---
    title = driver.find_element(By.TAG_NAME, "h2").text
    assert len(title) > 0, "Campaign title missing"
    print(f"🟢 Campaign title: {title}")

    business = driver.find_element(By.XPATH, "//p[contains(., 'Business:')]").text
    print(f"🟢 Business info found: {business}")

    # --- Step 5: Progress bar and funding info ---
    progress_bar = driver.find_element(By.CSS_SELECTOR, ".progress-bar")
    percent = progress_bar.get_attribute("data-target")
    print(f"📊 Progress: {percent}%")

    status_text = driver.find_element(By.XPATH, "//li[contains(., 'Status:')]").text
    print(f"🟡 {status_text}")

    # --- Step 6: Click "Invest in this Campaign" button ---
    try:
        invest_button = wait.until(
            EC.element_to_be_clickable((By.XPATH, "//a[contains(text(), 'Invest in this Campaign')]"))
        )
        invest_button.click()
    except:
        print("⚠️ Invest button not found or not clickable")

except Exception as e:
    print(f"❌ Error: {e}")
finally:
    driver.quit()
```

- 🔑 Navigating to login page...
- ✅ Logged in as investor_user1234
- ✅ Opened campaign details page
- ✅ Campaign card loaded
- 🟢 Campaign title: servo motor
- 🟢 Business info found: Business: micro processor
- 📊 Progress: 0.00%
- 🟡 Status: Open
- 🟢 Found 'Invest in this Campaign' button – clicking...
- 🔗 Redirected to Investment Page successfully!
- ✅ Investment form loaded (amount field visible)
- 🔗 Returned to campaign details page
- 📄 Campaign Updates section found
- 🔵 No updates posted yet – verified
- ✅ Investor flow tested successfully!
- 🔴 Browser closed

Portfolio:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

# --- Setup ---
driver = webdriver.Chrome()
driver.maximize_window()
wait = WebDriverWait(driver, 15)

# --- Step 1: Go to Login Page ---
driver.get("http://127.0.0.1:8000/login/") # Change base URL if needed

(variable) username_field: WebElement
username_field = wait.until(EC.visibility_of_element_located((By.ID, "username")))
password_field = driver.find_element(By.ID, "password")

username_field.send_keys("investor_user1234") # your investor username
password_field.send_keys("TestPassword123") # your investor password

login_button = driver.find_element(By.XPATH, "//button[contains(text(),'Login')]")
login_button.click()
print("✅ Logged in successfully as Investor")
time.sleep(2)

# --- Step 4: Go to Investment Opportunities page ---
driver.get("http://127.0.0.1:8000/portfolio/") # change URL to your actual invest page
wait.until(EC.presence_of_element_located((By.XPATH, "//h2[contains(text(),'Your Portfolio')]")))
print("✅ Investment Opportunities page loaded")

# --- Finish ---
time.sleep(2)
driver.quit()
```

10.5 Bug Reports and Fixes

Bug ID	Description	Severity	Fix Implemented
B01	Login with incorrect password did not show proper error	Medium	Added error message display logic
B02	Project cards misaligned on mobile screen	Low	Updated CSS grid layout
B03	Form submission without CSRF token	High	Enabled CSRF token in Django templates
B04	Dashboard did not update after investment	High	Fixed database query and AJAX refresh

10.6 Lessons Learned from Testing

- ✓ Testing must be done after each module is completed
- ✓ Automation (Selenium) saved time in repeated login and form testing
- ✓ Security testing is critical due to financial transactions
- ✓ Proper error handling enhances user trust and usability

11. Project Management

11.1 Gantt Chart (Timeline, Phases, and Milestones)

Items	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7
Project Idea & Planning							
Requirement Analysis							
UI/UX Wireframing (Figma)							
Database & Backend Setup							
Front-End Development							
Integration & Testing							
Final Report & Presentation							

11.2 Task Division and Team Responsibilities

Team Member	Responsibilities
Shahriar Alam Shimul	Backend development (Django), Database modelling, Admin Panel
Salman Nur Ramim	Frontend development, HTML/CSS/JS design, Dashboard UI
Tahsin Sheikh	API integration, Messaging system, Testing and debugging

11.3 Resource Allocation

Resource	Purpose
Laptops & Internet	Development and communication
GitHub Repository	Version control and code management
Figma	UI/UX design and wireframes
Google Forms	Survey data collection

Resource	Purpose
Django Framework	Backend logic and authentication
MySQL Database	Storing users, projects, investments

11.4 Risk Assessment and Mitigation

Risk	Impact	Mitigation Strategy
Delay in development	Medium	Follow weekly timeline & task distribution
Data security breach	High	Implement encryption, HTTPS, secure login
Payment gateway failure	High	Use sandbox mode for testing, fallback options
SME fraud or fake data	High	Implement KYC, admin verification
Team communication issues	Medium	Weekly meetings, GitHub for tracking updates

12. Results and Discussion

12.1 System Demonstration Summary

The developed system **Sohoj Biniyog** successfully demonstrates a working prototype of a halal investment platform for SMEs. Key functionalities tested include:

- User registration and login with role selection (Investor/Entrepreneur/Admin)
- SME proposal submission and admin approval process
- Investment feature with profit-sharing contract generation
- Real-time dashboards for both investors and SME owners
- Messaging system for direct communication between parties

12.2 Key Functionalities Achieved

Functionality	Status
User authentication (login, registration, roles)	Completed
SME project proposal and admin verification	Completed
Investment and contract generation system	Completed
Dashboards for investors and entrepreneurs	Completed

Functionality	Status
Messaging and notifications	Completed
Payment gateway integration (simulated using API structure)	Prototype Level
Shariah-compliant profit-sharing logic	Integrated conceptually

12.3 Comparison with Existing Systems

Features	Traditional Banks	Crowdfunding Platforms	Sohoj Biniyog
Shariah-Compliant	✗	✗	✓
Interest-Free (Profit Sharing)	✗	✓ (donation-based only)	✓
SME-Focused Funding	Limited	Partially	✓
Investor Dashboard	✗	Limited	✓
Admin Approval and Verification	✓	✗	✓
Real-Time Transparency	Limited	✗	✓
Minimum Investment Requirement	High	Low	Very Low (₹500–₹1000)

12.4 User Feedback

From surveys and interviews:

- 85% of users liked the idea of halal investments
- Majority showed interest in investing small amounts monthly
- Trust and transparency were considered critical by most participants
- Some users were concerned about **fraud and fake SMEs**
- Users suggested **mobile app and live investment tracking** for future versions

13. Conclusion and Future Work

13.1 Summary of Achievements

The *Sohoj Biniyog* project successfully demonstrates a working prototype of a **halal investment platform** that connects investors with SME entrepreneurs.

Throughout this project, the team achieved the following:

- Designed a user-friendly web application using Django, HTML, CSS, and MySQL
- Implemented secure user authentication with role-based access (Investor / Entrepreneur / Admin)
- Enabled SME proposal submission and admin approval
- Built an investment and profit-sharing model based on Islamic finance principles (Mudarabah / Musharakah)
- Developed dashboards for both investors and SME owners with real-time data
- Collected real survey and interview data to validate the system's need and user interest
- Ensured transparency, usability, and operational feasibility

13.2 Limitations of the Current System

Despite the successful implementation, the system still has a few limitations:

Limitation	Description
No real payment gateway integration	Current system uses mock API structure, not real transactions
Limited security implementation	Advanced encryption and blockchain-based transparency are not yet implemented
Web-only platform	No mobile app currently available
No legal certification	Not registered as a financial organization in Bangladesh
Limited user base testing	Only prototype testing with small group users was performed

13.3 Future Enhancements and Research Directions

- To improve and scale the project, the following enhancements are proposed:
Mobile Application for Android & iOS using Flutter/React Native
- **Real Payment Gateway Integration** (bKash, Nagad, SSLCommerz, Islamic Bank APIs)

- **AI-Based Investment Recommendation System** for investors
Blockchain or Smart Contracts for transparent, unchangeable profit records
- **Automated Zakat & Profit Calculation** following Islamic principles
Global Expansion to include diaspora investors from abroad
- **Legal Approval and Registration** as a certified Islamic fintech company
Fraud Detection System using KYC verification, AI-based anomaly detection

14. References

1. Google LLC, *Google Forms: Online forms to get insights quickly*. [Online]. Available: <https://workspace.google.com/products/forms>
workspace.google.com.
2. Django Software Foundation, *Django – The web framework for perfectionists with deadlines*. [Online]. Available: <https://www.djangoproject.com>
[djangoproject.com](https://www.djangoproject.com).
3. Figma Inc., *Think bigger. Build faster*. [Online]. Available: <https://www.figma.com> [figma.com](https://www.figma.com).
4. SeleniumHQ, *Selenium WebDriver*. [Online]. Available: <https://www.selenium.dev> [selenium.dev](https://www.selenium.dev).
5. B. Hossain et al., “Exploring the Potential of Islamic Crowdfunding as an Alternative Financing Source for Entrepreneurs in Bangladesh,” *Int. J. Sci. Bus.*, vol. 41, no. 1, pp. 56–68, 2024 [researchgate.net](https://www.researchgate.net).
6. Ethis Global, “*Ethical Crowdfunding: The Road to Financial Inclusivity*,” Ethis Blog, June 16, 2023 ethis.co.
7. FundingSouq Editorial Team, “*The Basics of Shariah Compliant Profit & Loss Sharing Models*,” FundingSouq Blog, Jan. 25, 2025 [fundingsouq.com](https://www.fundingsouq.com).

16. Appendices

16.1 Survey and Interview Questionnaires

Survey Questionnaires

Survey link:

<https://docs.google.com/forms/d/e/1FAIpQLSfAiYievoA-oTadrmZZVBV0ZFKNVW4ld2Pzv2f0UPqyqnkfaA/viewform?usp=header>

Demographic Section:

- Age: [Under 18], [18–25], [26–35], [36–45], [46+]
- Occupation: [Student], [Business Owner], [Service Holder], [Other]

Multiple Choice:

5. Have you ever invested in any platform?
 - Yes / No
6. Which investment range do you feel comfortable with?
 - Less than ₪500, ₪500 – ₪1,000, ₪1,000 – ₪5,000, More than ₪5,000
7. How often would you prefer to invest?
8. What motivates you to consider Halal investments?
 - Ethics, Profit, Simplicity, Religious Value

Likert Scale (1=Strongly Disagree, 5=Strongly Agree):

How much do you agree with the following statements?

- I am comfortable using mobile/web apps for financial transactions.
- I am interested in investing in ethical (halal) businesses.
- I trust platforms that show transparency and real-time updates.
- I would prefer investing in small, verified local businesses (SMEs).
- I am more likely to invest if the platform provides proper Islamic guidance.

Open-ended:

- What features or services would you like to see in a small investment platform like Sohoj Biniyog?
- What concerns would stop you from using such a platform?

Interview Questionnaires

Semi-Structured Questions:

6. What is your first impression of a platform like “Sohoj Biniyog”?
7. What would motivate you to trust a platform like "Sohoj Biniyog"?
8. Which tech stack (language/frameworks/tools) would you personally use to build a scalable MVP for this platform, and why?
9. What would be your biggest concern if you were investing in a project like this — from both a tech and business angle?

10. What advice would you give to a student team trying to turn this project into a real product or startup?

16.2 Figma Design Screenshots or Link

Figma Link: <https://www.figma.com/design/KEOynrSEfg3BPjPicJp78E/Sohoj-Biniyog?node-id=0-1&t=oZiEGmRORZLsshmH-1>

16.3 Selenium Test Logs

Selenium Testing Video:

https://drive.google.com/file/d/1cHsG5V5Nlc5bpwUidQwKFSMqI-U0VMwC/view?usp=drive_link

```
✓ Scrolled through the page
✓ Selected role: Business Owner
✓ Filled in registration details
✓ Clicked Register button
```

```
✓ Scrolled through the page
✓ Selected role: Investor
✓ Filled in registration details
✓ Clicked Register button
```

```
✓ Logged in as Investor
✓ Opened campaign investment page
✓ Entered investment amount: 65000
✓ Clicked 'Invest Now' button
```

```
🔗 Navigating to login page...
✓ Logged in as investor_user1234
✓ Opened campaign details page
✓ Campaign card loaded
● Campaign title: servo motor
● Business info found: Business: micro processor
📊 Progress: 0.00%
🔥 Status: Open
● Found 'Invest in this Campaign' button - clicking...
🔗 Redirected to Investment Page successfully!
✓ Investment form loaded (amount field visible)
🔗 Returned to campaign details page
📄 Campaign Updates section found
📄 No updates posted yet - verified
✓ Investor flow tested successfully!
🔥 Browser closed
```

```
✓ Investment opportunities page loaded
✓ Logged in successfully as investor
```

16.4 GitHub Repository Link and Commit Summary

- Repository: <https://github.com/Tahsin22201243/SohojWeb>
- <https://github.com/Tahsin22201243/SeleniumTesting>

Commits				
main			All users	All time
Commits on Oct 28, 2025				
Merge branch 'main' of https://github.com/Tahsin2201243/SohoWeb			6c548b4	
Initial commit			988ba07	
Commits on Oct 24, 2025				
Update README.md	Verified		ec5a05a	
Update README.md	Verified		0983464	
Update README.md	Verified		37af05d	
Create README.md	Verified		857d64b	
Commits on Oct 20, 2025				
Selenium testing done for all the pages of the project			95d8052	
ui upgrade, backend modified, updated views logic			e743a05	
Commits on Sep 29, 2025				
Login Registration added			2ed2785	
Commits on Sep 28, 2025				
Initial project commit			ead597c	
Commits on Sep 25, 2025				
Modified CSS			4527f85	
Commits on Sep 22, 2025				
Navbar Updated			ca5e78d	
Frontend Modified			47ab298	
Commits on Sep 21, 2025				
html files created			92755c7	
Commits on Sep 20, 2025				
Backend for some pages			355acaf	
requirement.txt file created			7936bda	
Commits on Sep 16, 2025				
Initial commit				

16. CEP Mapping of the Project

1. Knowledge Profile (K's) Mapping

K's Attributes How K's Are Addressed Through Our Project CO PO				
K Code	Attribute	How K's Are Addressed Through Our Project	Related COs	Related POs
K4	Specialist Knowledge	Applied web engineering and software concepts: Django backend (models, views, ORM), REST-like endpoints, HTML/CSS/JS frontend, authentication & session management, database design (SQLite), basic security (CSRF handling).	CO1, CO2	PO3, PO5

K5	Engineering Design	System architecture and UI/UX design: multi-role flows (Business Owner, Investor), campaign data model, investment transaction flow, funding-progress logic, and design of automation test architecture (Selenium test harness).	CO2, CO3, CO 4	PO3, PO5,
K6	Engineering Practice	Practical software implementation and tooling: Django project setup, static/media config, frontend & responsiveness, webdriver-manager usage, and deployment preparation (cloud notes).	CO3, CO 4, CO5	PO4, PO5, PO6

2. Complex Engineering Problems (P's) Mapping

P's Attributes How P's Are Addressed Through Our Project CO PO				
P Code	Attribute	How P's Are Addressed Through Our Project	Related COs	Related POs
P4	Practical Implementation	Integrated modules: user auth, campaign creation, investment transactions (atomic updates), progress calculations, admin monitoring and frontend display; configured static files and media in Django.	CO2, CO3	PO4, PO5
P5	Experimentation & Testing	Developed and executed automated tests: Selenium scripts for login, create campaign, invest flow, logout; used explicit waits, session handling and test DB resets to ensure reliability.	CO3, CO4	PO4, PO9
P6	Prototyping & Validation	Validated system behavior via test results, manual exploratory testing, and demo scenarios (investor browsing, funding reaching target, admin checks)..	CO4, CO5	PO5, PO6

3. Complex Engineering Activities (A's) Mapping

A's Attributes How A's Are Addressed Through Our Project CO PO				
A Code	Attribute	How A's Are Addressed Through Our Project	Related COs	Related POs

A3	Teamwork & Collaboration	Collaborative development on GitHub with feature branches, pull requests, issue-tracking, role assignments (frontend, backend, testing), joint debugging and integration sessions.	CO4	PO9 , PO1 1
A5	Ethics & Sustainability	Followed ethical guidelines for user data (no sensitive storage), proposed secure payment integration, privacy-aware notifications, responsible disclosure of recorded demo links.	CO3 , CO5	PO6, PO7